Experiment 6: Missionaries and Cannibals Problem

CSA1717 – Artificial Intelligence

Name: Hemalakshmi H

Reg.No.: 192571049

Aim:

To solve the Missionaries and Cannibals problem using Breadth-First Search (BFS) by safely transporting 3 missionaries and 3 cannibals across a river using a boat that can carry one or two persons, ensuring missionaries are never outnumbered by cannibals on either bank.

Algorithm:

Step 1: Define State Representation

Represent each state as a tuple: (M_left, C_left, Boat_side)

- M_left: missionaries on left bank
- C_left: cannibals on left bank
- Boat_side: 1 = boat on left, 0 = boat on right

Step 2: Define Validity Rules

A state is valid only if:

- No count is negative
- Missionaries are not outnumbered by cannibals on either bank
- Boat moves 1 or 2 persons

Step 3: Generate Successor States

Possible moves:

- (1 missionary)
- (2 missionaries)
- (1 cannibal)
- (2 cannibals)
- (1 missionary + 1 cannibal)

Apply moves depending on boat side.

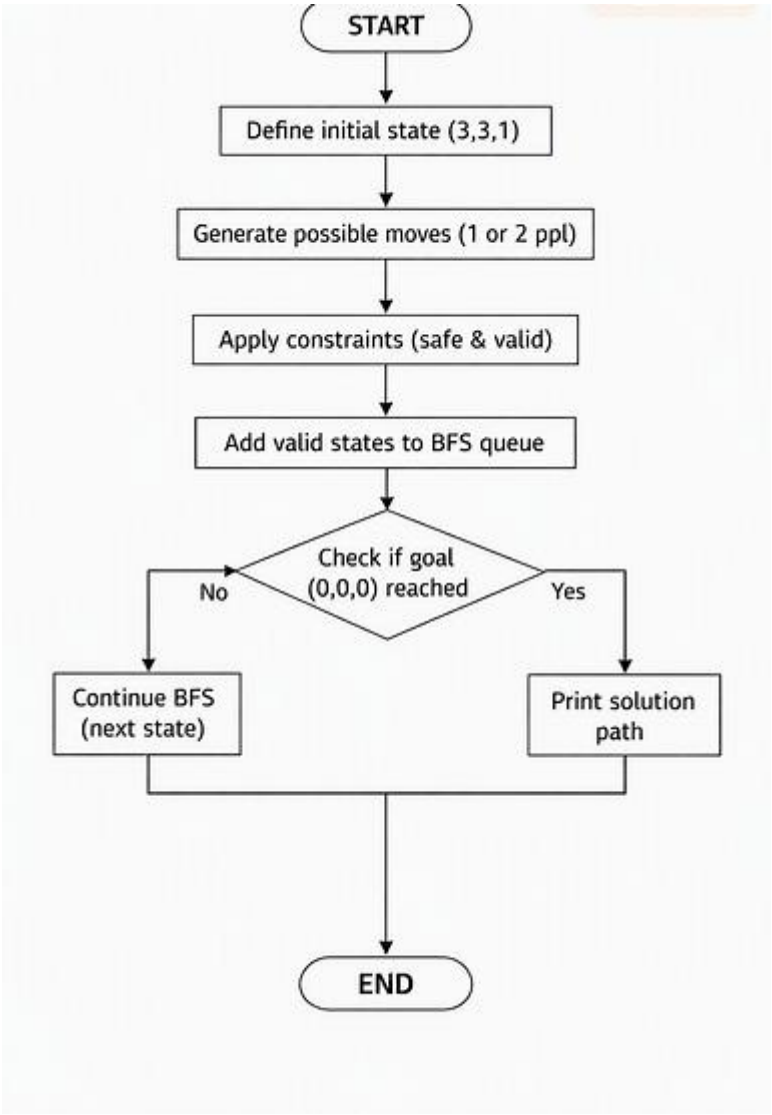
Step 4: Use BFS

- Start from initial state (3,3,1)
- Explore all valid next states
- Avoid repeated states
- Stop when reaching goal (0,0,0)

Step 5: Output Solution Path

Print the sequence of states showing safe transitions.

Flowchart:



Source Code:

```
# Missionaries and Cannibals Problem
# Using Breadth-First Search (BFS)

from collections import deque

# State format: (M_left, C_left, boat_side)
# boat_side = 1 → boat on left bank
# boat_side = 0 → boat on right bank

def is_valid(state):
    M_left, C_left, boat = state
    M_right = 3 - M_left
    C_right = 3 - C_left

    # No negative values allowed
    if M_left < 0 or C_left < 0 or M_right < 0 or C_right < 0:
        return False

    # Missionaries must not be outnumbered on either side
    if (M_left > 0 and M_left < C_left):
        return False
    if (M_right > 0 and M_right < C_right):
        return False

    return True


def get_successors(state):
```

```
    M_left, C_left, boat = state
    successors = []

    # Possible boat moves: (missionaries, cannibals)
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

    for M, C in moves:
        if boat == 1:  # boat on left bank → move people to right
            new_state = (M_left - M, C_left - C, 0)
        else:          # boat on right bank → move people to left
            new_state = (M_left + M, C_left + C, 1)

        if is_valid(new_state):
            successors.append(new_state)

    return successors



def bfs(start, goal):
    queue = deque([(start, [start])])
    visited = set([start])

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path

        for next_state in get_successors(state):
```

```
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [next_state]))

    return None



# Initial and goal states
start_state = (3, 3, 1)
goal_state = (0, 0, 0)

solution = bfs(start_state, goal_state)

print("\n=== Missionaries & Cannibals Solution ===")
if solution:
    for step in solution:
        print(step)
else:
   print("No solution found.")
```

Output:

```
========== RESTART: C:/Users/hemal/OneDrive/Documents/SIMA

=== Missionaries & Cannibals Solution ===
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
>>
```

Result:

The Missionaries and Cannibals problem was solved using BFS. Each state is represented as (M_left, C_left, Boat_side).

The algorithm successfully found a safe sequence of moves that transfers all 3 missionaries and 3 cannibals from the left bank to the right bank without ever allowing cannibals to outnumber missionaries on either side.

The final output sequence ends with: